SCOUT: A Single-Round, Fixed-Budget Probe for Continuous Reconciliation in Heterogeneous Replication

Min Wu
Hangzhou Dianzi University
wumin@hdu.edu.cn

Abstract

Continuous data replication requires repeated source–target validation. In heterogeneous replication, the two sides cannot rely on shared internal checksums or repair primitives, so validation must proceed through a black-box replica interface. Yet precise verification in every round incurs sustained cost proportional to data scale, even though most rounds reveal no actionable discrepancy.

This paper presents SCOUT, a single-round, fixed-budget diagnostic probe for continuous reconciliation over black-box replicas. From a single sketch exchange, SCOUT estimates discrepancy size, predicts whether narrow verification is likely to pay off, and identifies data segments that merit follow-up verification. These decisions are derived from a shared energy statistic whose interpretation remains stable under side imbalance. By coupling each segment to a contiguous observation interval, SCOUT turns concentrated faults into localized energy peaks that can be compressed through sliding-window aggregation. When the signal is too diffuse for reliable compression, SCOUT abstains and defers to a fallback verification path.

Across same-budget baselines and large-scale YCSB Zipfian workloads, SCOUT reduces candidate cost by nearly an order of magnitude under concentrated faults, keeps sizing error within 3% even under severe side imbalance, and achieves low false-alarm and false-rejection rates for verification gating.

1 Introduction

We address continuous reconciliation in heterogeneous data replication, where source and target replicas must be repeatedly validated despite running on different storage engines, such as MySQL to TiDB or Oracle to ClickHouse.

As in any replication pipeline, replicas can become temporarily or persistently inconsistent. In heterogeneous deployments, however, validation is harder because the two engines typically do not share internal checksum structures, reconciliation metadata, or repair primitives. Even where such internals are exposed, their formats, granularities, and version-specific behaviors evolve independently, making a stable cross-engine interface difficult to maintain. Validation must therefore operate through a black-box replica interface: sequential enumeration of comparable items without dependence on engine-specific internals.

Heterogeneous replication also introduces additional sources of residual divergence beyond ordinary infrastructure faults. Even when a deployment-defined comparison contract normalizes documented incompatibilities, independently evolving engines can still disagree on version-specific behavior, corner cases, or other data-dependent effects. Because prevention cannot eliminate this residual class completely, validation must remain an independent line of defense. In deployments where heterogeneous systems operate side by side for extended periods, repeated reconciliation is therefore an operational requirement rather than a one-time migration check. Its cost must therefore be assessed per reconciliation round. The goal is not to remove the unavoidable local scan, but to ensure that normal rounds incur only fixed diagnostic overhead, while anomalous rounds pay additional cost only in proportion to the scope of suspected discrepancy. We express this objective through the following idealized per-cycle cost model:

$$\mathrm{Cost}_{\mathrm{cycle}} \;=\; C_{\mathrm{scan}} \;+\; C_{\mathrm{probe}} \;+\; \mathbf{1}[\hat{D} > \tau] \;\cdot\; C_{\mathrm{verify}}(|\mathcal{C}|),$$

where C_{scan} is the local sequential-scan cost shared by external validation methods operating through a black-box replica interface; C_{probe} is the fixed-size digest overhead—summary maintenance, exchange, and decoding; the indicator is triggered when the estimated discrepancy $\hat{D}$ exceeds an actionable threshold τ; and $C_{\mathrm{verify}}(|\mathcal{C}|)$ is the downstream verification cost, proportional to the candidate scope $|\mathcal{C}|$ rather than the full segment count P.

Achieving this cost model in a single round requires a fixed-size digest whose cost remains decoupled from data scale and whose construction relies only on a black-box replica interface. Existing mechanisms do not satisfy this combination of requirements. Multi-round approaches, including Merkle-

based anti-entropy and set reconciliation, can progressively localize differences, but they depend on iterative interaction. By contrast, single-round summaries can often support either discrepancy estimation or candidate prioritization under a fixed budget, but not both within the same exchange. They also cannot tell when candidate generation should be withheld—that is, when the candidate set is likely to approach the full segment count. Naively composing separate sketches does not close this gap. Splitting the budget weakens both components. Moreover, the resulting estimates need not share a common statistical basis, so scale assessment, admission, and candidate ranking can produce inconsistent conclusions.

We present SCOUT, a single-round diagnostic probe for continuous reconciliation under a fixed digest budget. Rather than recovering all differences, SCOUT decides how large the discrepancy is, whether narrowed-scope verification is worthwhile given the discrepancy shape, and where downstream verification should focus. From one sketch exchange it produces three corresponding outputs: a discrepancy-scale estimate (*sizing*), an admission decision on whether candidate compression is economical (*gating*), and a compressed candidate set for precise downstream verification (*triage*).

SCOUT relies on two ideas. First, it preserves locality in the observation space, so that concentrated discrepancies appear as localized peaks that can be compressed by sliding-window aggregation. Second, it uses a de-drifted variance statistic that removes first-order side imbalance, allowing the same probe to support stable estimation and admission even under highly asymmetric discrepancies. When the evidence is too diffuse or too weak to support reliable compression, SCOUT explicitly abstains and defers to a conservative fallback.

We make three contributions. (1) We formulate the problem as a single-round, fixed-budget diagnostic contract for heterogeneous replication, and use this contract to identify the structural gap in existing approaches (§2). (2) We design SCOUT, a unified probe in which scale estimation, admission control, and candidate triage share a single de-drifted signal. A locality-preserving observation geometry enables candidate compression for concentrated faults. The same de-drifted statistic keeps all three outputs stable under arbitrary side imbalance (§4). (3) We evaluate SCOUT against same-budget baselines and YCSB Zipfian workloads (§5). The results show substantial candidate-scope reduction under concentrated faults, stable sizing under drift, and reliable shape-based gating with low false-alarm and false-rejection rates.

2 Problem Formulation

System Model and Constraints We consider two replicas A and B that must remain continuously consistent and may reside on heterogeneous systems. The key space is partitioned into P logical units, called *segments*, by a deterministic segmenter shared by both sides (e.g., consistent hashing or prefix mapping), so that the same key is assigned to the same segment without accessing engine-specific metadata.

In each reconciliation round, A and B construct and exchange a fixed-size probe, then decode it locally to obtain diagnostic signals that guide downstream precise verification and repair. The probe is diagnostic rather than restorative: it does not recover all differing items, but summarizes their scale, shape, and likely scope.

In heterogeneous deployments, agreement on key equality and segmentation is provided by a deployment-defined comparison contract: a deterministic normalization layer plus a shared segmenter. SCOUT assumes this contract and diagnoses residual divergence with respect to it; defining the contract itself is outside the scope of this paper. The only interface requirement is sequential enumeration of comparable items—raw keys or normalized identifiers that incorporate value or version digests—with incremental hash updates and no dependence on engine-specific internals. Because the observation budget is much smaller than P, aliasing is unavoidable, and the probe cannot guarantee useful candidate compression under all discrepancy patterns. This limitation motivates the admission mechanism defined below.

Diagnostic Primitives We now define the three output types precisely.

Sizing (scale estimation) estimates the global difference scale. Let $D^+ = |A \setminus B|$ and $D^- = |B \setminus A|$; the total discrepancy size is $D = D^+ + D^-$. Sizing always emits $\hat{D}$, regardless of gating verdict or discrepancy shape. In normal rounds, $\hat{D}$ stays near zero, with small residual values attributable to estimation noise, allowing the system to skip downstream verification; in anomalous rounds it provides a severity signal independent of the gating verdict.

Gating (admission control) determines whether the current round's discrepancy shape supports economical candidate compression. It emits a three-valued verdict: `notgate` admits candidate generation, `gate` diverts to a conservative fallback, and `unknown` (*insufficient observability*) signals that the probe's fixed-budget observation lacks sufficient support for a reliable shape verdict. In this case, downstream policy combines $\hat{D}$ and other observables to determine the round's disposition.

Triage (candidate scope) is executed when gating produces a `notgate` verdict. It outputs a candidate segment set $\mathcal{C} \subseteq \{1, \ldots, P\}$ whose objective is to minimize downstream scan scope, typically measured by $|\mathcal{C}|$ or candidate fraction, subject to coverage of serious discrepancies.

Together, these define the probe's interface contract: from one digest exchange the system obtains $(g, \hat{D}, \mathcal{C})$—an admission verdict, a discrepancy-scale estimate, and a candidate segment set produced when $g =$ `notgate`. Precise localization and repair within $\mathcal{C}$ are delegated to downstream mechanisms such as Merkle trees or IBLTs.

Drift: Side Imbalance The distribution of differing keys across the two sides is often asymmetric. Under bidirectional replication, for example, both sides may continue receiving writes while update propagation proceeds unevenly, causing unsynchronized keys to accumulate predominantly on one side. We refer to this systematic side imbalance as *drift*, parameterized by a scalar $q \in [0,1]$ denoting side A's share of the total differing keys ($q = 0.5$ means the differences are evenly split, while $q \to 0$ or 1 means they reside almost entirely on one side). A practical probe should remain accurate across this range of side splits rather than depending on a particular allocation of differences between the two sides.

3 Related Work

Precise localization and recovery Merkle trees and the anti-entropy family [1–4] locate and repair inconsistencies through iterative drill-down, but each round requires a cross-replica exchange; in cross-datacenter settings the interaction cost is amplified by RTT [5, 6]. Set reconciliation [7] and IBLT variants [8, 9] can recover all differing items in a single round only below a decodable capacity threshold [10]; later refinements such as PBS [11] and rateless IBLT [12] reduce this rigidity but still require either prior size estimates or multi-round adaptation. Recent work further improves IBLT space efficiency [13], robustness trade-offs [14], and parameterless guarantees [15]. In the continuous-validation setting we study, these mechanisms serve as downstream verifiers; the missing piece is a fixed-budget front-end that decides *whether* and *where* to invoke them.

Single-round summaries and sketches The first path targets global discrepancy sizing. Bloom-filter-family structures [16–19] and HLL-style estimators [20, 21]—deployed in production as Redis HyperLogLog [22] and Apache DataSketches [23]—provide compact cardinality primitives; the Strata estimator [8] layers sampling over IBLTs to estimate difference size across scales; SetSketch [24] and the Odd Sketch [25] unify cardinality and similarity estimation in a single structure. All can emit a sizing signal from small summaries; localization and admission are outside their design goals.

The second path targets per-segment ranking. Heavy-hitter sketches such as CountSketch [26] and Count-Min [27, 28] are fixed-budget frequency-estimation primitives widely deployed in streaming and telemetry settings [29–33]. In our setting, their per-item ranking capability provides a natural fixed-budget primitive for candidate generation: treating each segment as an item and its difference count as a frequency yields a top-K candidate set ranked by estimated divergence intensity. We adopt this adaptation as baseline B2 in §5.5. Candidate quality depends on the separation margin between dirty segments and background noise; we quantify this dependence empirically in §5.5. Learned variants [34] improve per-item accuracy under stable frequency priors but do not address the admission-control role studied here. A ranking sketch can always return a top-K list, but cannot tell when doing so is uneconomical or when fallback would be preferable.

Another fixed-budget alternative is a fixed-depth hierarchical bucket tree that provides coarse localization through structured aggregation (baseline B3 in §5.5). Its bucket granularity leads to candidate expansion in discrete jumps, making the trade-off between localization and candidate size more coarse-grained; we examine this behavior empirically in §5.5.

For a single-round fixed-budget front-end probe, these capabilities—scope, scale, and admission—have not previously been unified in a single structure. Table 1 summarizes how representative approaches map to these outputs in this setting.

Table 1: Representative approaches mapped to the three diagnostic outputs studied in this paper, in the single-round fixed-budget setting with a black-box replica interface. ✓ = supported; ✗ = not supported; △ = partial or fragile.

	Scope	Scale	Triage admission
Bloom / HLL family	✗	✓	✗
Strata estimator	✗	✓	✗
CountSketch / CM (ranking, B2)	✓	△	✗
Hierarchical bucket tree (B3)	✓	△	✗
Naive composition	✓	✓	△
SCOUT	✓	✓	✓

Group testing and spatially coupled designs Triage can be viewed as identifying a sparse anomalous set from limited aggregate measurements, a setting studied in group testing [10]. SCOUT's contiguous-support mapping draws on the observation that spatially coupled measurement designs can improve inference thresholds under budget constraints [35, 36].

Industrial practice Production systems commonly trade off between multi-round anti-entropy and coarse per-range checksum workflows [37–40]. In heterogeneous replication these tensions are sharper: the two sides often cannot share internal checksum structures or repair logic, leaving black-box summary exchange as the only portable interface [5, 6, 41, 42]. Industrial source–target validation tools—including `pt-table-checksum` [43], AWS DMS validation [44], GCP migration verification [45], and CockroachDB MOLT Verify [46]—universally perform exact row-level comparison whose cost scales with the full data footprint on every invocation. These tools target migration cutover or post-failure audit, where a one-time full scan is acceptable. In the continuous setting studied here, however, most rounds find no

actionable discrepancy, and the per-invocation cost of a full scan becomes the dominant overhead.

4 Design of SCOUT

4.1 Probe Structure and Diagnostic Pipeline

Probe structure SCOUT uses the same fixed-size probe structure at both replicas. Each replica maintains V independent *views*; each view is a linear array of L *sections*, each section containing m_0 cells. Each cell stores a 32-bit signed counter, so each view contains $M = L \cdot m_0$ cells. Across all V views, the probe occupies $4VLm_0$ bytes (e.g., $\sim$512 KiB at V=2, L=2048, m_0=32), fixed at configuration time and independent of data scale P.

A two-level mapping (Fig. 1) writes each key into this array: the key's segment determines a contiguous window of W sections on the section axis, and a second hash selects k cells within that window. The contiguous-window structure is central to candidate compression and is analyzed in §4.2. For analysis, both mapping stages are modeled as uniform and independent hashing; hash robustness is verified in Supplement S8.

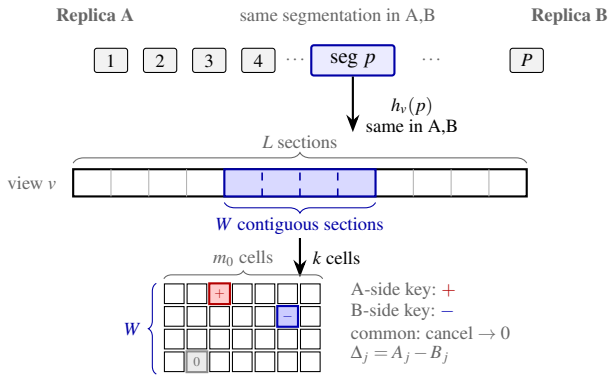


Figure 1: Two-level mapping in one view: segment p is hashed to W contiguous sections, then each key is hashed to k cells within that window.

Pipeline Each reconciliation cycle executes four stages; the first three are fixed-cost, while the fourth adapts to the observed evidence.

(1) Build. Each replica sequentially scans its local key space; every item is inserted into the probe array via the two-level mapping described above.

(2) Exchange & Diff. The two replicas exchange their probes once; each side subtracts cell-wise to obtain the difference array $\Delta_j = C_j^{(A)} - C_j^{(B)}$. Because both replicas apply identical hash mappings to each item, any item present on both sides produces equal updates to the same probe cells. These contributions cancel under cell-wise subtraction, so Δ is driven solely by items that differ between the two sides: items in $A \setminus B$ contribute $+1$ signed updates, while those in $B \setminus A$ contribute -1.

(3) Decode. For every section s in each view, the decoder computes the first moment and the raw second moment:

$$\text{net_sum}_s = \sum_{j \in s} \Delta_j, \ \text{sq_sum}_s = \sum_{j \in s} \Delta_j^2,$$

then constructs the **de-drifted energy**:

$$E_s \ = \ \text{sq_sum}_s \ - \ \frac{(\text{net_sum}_s)^2}{m_0}.$$

E_s is the unnormalized sample variance of the differential counts within section s. It serves as the *common signal* from which all three outputs are derived: triage consumes the spatial distribution of $\{E_s\}$ (where energy concentrates), sizing consumes the total mass $\sum_s E_s$ (how much energy exists), and gating consumes the support dispersion of $\{E_s\}$ (over how many sections energy spreads). All three readings come from the same energy spectrum without budget partitioning. The subtraction of $(\text{net_sum}_s)^2/m_0$ removes the first-order sign bias that would otherwise dominate the raw second moment under side imbalance, so E_s captures fluctuation energy rather than drift-induced offset (note that $E_s \geq 0$ by the Cauchy–Schwarz inequality). The full derivation is given in Supplement S4.1.

(4) Decide. Each diagnostic output reads the same section-level energy spectrum but fuses information across the V views differently. Sizing computes a per-view scale estimate and averages across views. Gating evaluates the concentration metric and occupancy on the energy spectrum (§4.4). Triage retains per-view window rankings and intersects the resulting candidate supersets to suppress view-dependent false inclusions.

When all sections in a segment's mapped window carry zero signal (E_s=0 and net_sum$_s$=0) in any view, the segment can be excluded from downstream verification, because any diff key would have left a non-zero cell update within the window (Supplement S4.1.2).

Sizing emits $\hat{D}$ (§4.3); $\hat{D} \approx 0$ terminates the round without downstream verification. Otherwise, gating tests K_{eff}/L and occupancy (§4.4). On `notgate`, triage produces $\mathcal{C}$ via sliding-window aggregation and cross-view intersection (Fig. 2). On `gate`, the system suppresses candidate generation and defers to a conservative fallback—e.g., full-scope exact verification or a coarse per-range checksum pass over all P segments—whose cost scales with P rather than $|\mathcal{C}|$. On `unknown`, the probe withholds only a shape-based verdict; downstream policy may still invoke capped triage or full-scope fallback based on $\hat{D}$ and other observables (§5.3).

Beyond the unavoidable single sequential scan over local items, SCOUT incurs communication proportional to the fixed probe size and decoding cost of $O(VLm_0)$. Table 2 summarizes the core probe parameters.

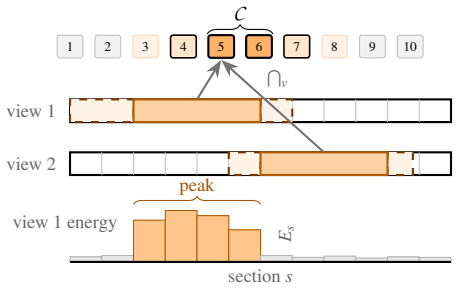


Figure 2: From energy peaks to candidate segments: sliding-window aggregation identifies high-energy regions on the section axis; reverse mapping and cross-view intersection yield the candidate set $\mathcal{C}$.

Table 2: Quick reference for SCOUT core parameters. Full symbol table in Supplement S9.

Symbol	Meaning
P	Total segments (10^4–10^7)
B	Concurrently dirty segments (unknown at runtime)
D	Symmetric difference size, $D = D^+ + D^-$
V	Views / independent hash channels (2–4)
L	Sections per view (1024–8192)
m_0	Cells per section (16–64)
W	Window width in contiguous sections (2–4)
k	Per-key update redundancy (1–3)

4.2 Observation Geometry for Candidate Compression

Triage operates on the energy spectrum $\{E_s\}$. Its ability to compress the candidate set depends on whether dirty-segment energy remains localized on the section axis. Localized energy produces separable peaks and enables candidate compression; dispersed energy blends into background noise and reduces triage to broad ranking.

Contiguous-interval mapping and sliding-window aggregation To preserve locality, SCOUT maps each segment to W **contiguous** sections on the section axis. Triage therefore scores *windows* rather than individual sections. For each starting position u, SCOUT computes

$$\Omega_u = \sum_{s=u}^{u+W-1} E_s,$$

the total energy in the width-W window beginning at u. This sliding-window aggregation pools the energy that a single segment deposits across its W adjacent sections, strengthening localized peaks while suppressing unstructured background noise—a width-W matched filter in signal-processing terms.

SCOUT then ranks all windows by Ω_u and keeps the top $t = \lceil r_{\text{top}} \cdot (L - W + 1) \rceil$ windows, where $r_{\text{top}} \in (0, 1]$ controls the candidate budget. Each selected window is then reverse-mapped to the segments whose supports overlap it (Fig. 2), producing a view-local candidate superset. The final candidate set $\mathcal{C}$ is obtained by intersecting these supersets across all V views, which suppresses false inclusions exponentially under approximately independent views while preserving true positives with high probability.

The compression effect above depends on *contiguity*: if the W target sections were scattered randomly instead of placed contiguously, the sliding window would lose its locality advantage and the scheme would reduce to independent per-section ranking (§5.2).

Scaling laws for candidate-set size Under the uniform-hashing model (§4.1), if concentrated differences produce separable signals, the expected candidate-set size after intersecting across V views is:

$$\mathbb{E}[|\mathcal{C}|] \approx B + (P - B) \cdot p_{\text{halo}}^V, \tag{1}$$

where B is the number of truly dirty segments and the second term captures the *halo* contribution: clean segments whose windows geometrically overlap with a selected dirty window are erroneously included in the candidate set. A clean segment's false-inclusion probability in a single view is

$$p_{\text{halo}} \approx \frac{t(2W-1)}{L} \quad (\text{when } t(2W-1) \ll L), \tag{2}$$

where $2W-1$ is the overlap neighborhood width of each selected window on the section axis. This framework implies three scaling laws: (i) cross-view intersection raises p_{halo} to the V-th power, making V the most effective lever against candidate inflation; (ii) the halo term grows with the number of clean segments, while larger L reduces per-view overlap and suppresses false inclusion; (iii) W improves signal aggregation but simultaneously inflates the halo via the $2W-1$ factor, so there is no uniformly optimal W (derivations in Supplement S2.1 and S2.2). All three laws require that dirty windows remain separable on the section axis; when L/B is too small, their supports overlap and candidate cost degrades sharply. This operating boundary is characterized empirically in §5.4 and modeled in Supplement S5.2.

4.3 Drift-Stable Sizing from the Shared Substrate

The de-drifted energy E_s (§4.1) removes the first-order sign bias that would otherwise cause sizing to overestimate by orders of magnitude under drift; we now derive the estimator built on it. Let $M = L \cdot m_0$ denote the total cell count of a single view. Under the same uniform-hashing model, the expected total de-drifted energy is:

$$\mathbb{E}\left[\sum_s E_s\right] = D \cdot k \left(1 - \frac{k}{M}\right).$$

Mean subtraction cancels the drift factor $(2q-1)^2$ exactly: both the squared-mean component of sq_sum and the correction term net_sum$^2/m_0$ scale quadratically with the mean sign bias $(2q-1)$, and their coefficients match, so after subtraction the expectation does not depend on q. The sizing estimator is

$$\hat{D} = \frac{\sum_s E_s}{k\left(1-k/M\right)},$$

where the denominator $1-k/M$ corrects for energy dilution from cross-key collisions within the same cell.

Two minor bias sources remain. First, E_s as a sample variance exhibits finite-sample shrinkage, removable by a Bessel correction factor $m_0/(m_0-1)$. Second, when differences are highly concentrated in a few sections the global uniformity approximation introduces a systematic underestimate of roughly 1–2%. After both corrections the measured relative bias converges to within approximately 3% across all tested configurations (§5.3).

Multi-view fusion provides additional robustness: averaging across approximately independent views drives the standard deviation down as $1/\sqrt{V}$, making V and m_0 the two main tuning knobs for sizing precision. Derivations covering drift cancellation, bias corrections, and multi-view fusion are given in Supplement S4.1 and S4.2.

4.4 Admission and Abstention: When Not to Triage

The gating decision rests on two observable quantities derived from the energy spectrum: how broadly the energy is spread, and whether the sketch carries enough statistical support to make that measurement reliable.

Concentration metric We measure energy dispersion by the inverse Herfindahl index

$$\frac{K_{\text{eff}}}{L} = \frac{1}{L}\frac{\left(\sum_s E_s\right)^2}{\sum_s E_s^2} \in [1/L, 1].$$

K_{eff}/L measures how many sections carry appreciable energy. It is invariant to overall discrepancy scale and to drift when computed from de-drifted energy. Under concentrated faults, energy resides in roughly BW sections, placing K_{eff}/L at the scale of BW/L, typically far below 1; under diffuse differences with sufficient occupancy it approaches 1. In the concentrated regime, K_{eff}/L also governs candidate inflation: a larger value requires a more generous truncation budget to preserve recall, widening halo overlap so that candidate cost grows as p_{halo}^V. K_{eff}/L therefore serves not only as a shape statistic but as a cost-oriented proxy for whether triage remains economical. Because concentrated and diffuse discrepancies are separated by roughly an order of magnitude in this metric, τ_{gating} needs only to fall between the two regimes rather than be tuned precisely (derivation in Supplement S4.5; empirical sensitivity in §5.3).

Observability guard-rail and three-state output The scale separation above requires sufficient statistical support in the energy spectrum. When occupancy is low, diffuse cases can collapse to support sizes comparable to concentrated ones, making K_{eff}/L unreliable. We define occupancy as the cross-view fraction of non-empty sections:

$$\text{occ} = \frac{1}{V}\sum_{v=1}^{V}\frac{1}{L}\sum_{s=1}^{L}\mathbf{1}\left[E_s^{(v)} > 0\right], \tag{3}$$

where $E_s^{(v)}$ denotes E_s evaluated on view v. SCOUT uses a three-state gating rule: $g = \texttt{unknown}$ if occ $< \tau_{\text{occ}}$; otherwise, $g = \texttt{notgate}$ if $K_{\text{eff}}/L \le \tau_{\text{gating}}$, and $g = \texttt{gate}$ otherwise. Neither threshold requires narrow tuning. τ_{gating} only needs to fall between the concentrated and diffuse regimes of K_{eff}/L, a gap that remains wide whenever occupancy is sufficient (§5.3). τ_{occ} only needs to exceed the typical concentrated-fault occupancy BW/L by a moderate margin to preserve scale separation within the observable region.

5 Evaluation

5.1 Setup, Baselines, and Metrics

The evaluation asks whether SCOUT can make reliable decisions across the discrepancy patterns that arise in continuous heterogeneous validation, without depending on any particular engine pair. We therefore organize the evaluation around concentrated faults, diffuse discrepancies, side imbalance (drift), and increasing discrepancy scale, rather than around a specific cross-engine deployment.

The evaluation proceeds in two stages. Parameterized synthetic workloads (§5.2–§5.4) isolate individual mechanisms and map operating boundaries. YCSB Zipfian workloads (§5.6) introduce segment-size skew and hotness imbalance to test robustness under more realistic data-distribution conditions. Synthetic workloads cover concentrated, diffuse, and mixed heavy-tail patterns with explicit sweeps over drift intensity; exact constructions are given in Supplement S1.1.

Baselines We compare three methods under the single-round fixed-budget constraint. B1 (per-segment checksum) maintains one checksum per segment; because its digest grows with P, it violates the fixed-budget constraint and serves only as a full-information reference rather than a same-budget baseline. B2 (CountSketch ranking) treats each segment as an item and outputs a top-K candidate set ranked by difference intensity. B3 (hierarchical bucket tree) maps segments to leaf nodes of a fixed-depth tree and drills down from root to leaves by selecting high-score buckets at each level; when a leaf bucket is selected, all segments it contains become candidates, producing step-wise cost jumps of P/b^d per bucket (Supplement Fig. S20). B2 and B3 represent the two most natural

fixed-budget single-round alternatives in this setting: unstructured ranking and structured hierarchical aggregation. All methods are aligned by digest memory in bytes at a primary tier of ∼512 KiB; each baseline is additionally tested at ∼2× memory (exact configurations in Supplement Table S19).

Metrics and operating-point selection The primary triage metric is the candidate fraction `cand_frac`$=|C|/P$. For triage comparisons, we sweep each method's internal tuning knob and report the best operating point that satisfies the target heavy-segment recall for the experiment, where `recall_big` measures recall over the truly heavily dirty segments in the workload instance. Where a candidate-budget cap is imposed, infeasible points are marked as **N/A**.

Gating quality is evaluated through false-alarm rate (FAR) and false-rejection rate (FRR) trade-offs. When the experiment includes abstention, we additionally report performance on decisive (non-`unknown`) samples. Sizing accuracy is reported as Std(rel_err), the standard deviation of the relative estimation error $(\hat{D}-D)/D$ across repetitions.

YCSB validation scales to $\sim 2\times 10^9$ records across P=100,000 segments with one-sided digest below 1 MiB. Details are in Supplement S1.3.

5.2 Spatial Coupling Enables Candidate Compression

This section validates the necessity of two key design choices in spatial coupling—window support ($W > 1$) and contiguous structure—for candidate compression.

Choice of window width Spatial coupling through overlapping windows ($W \geq 2$) improves ranking quality over per-section independent scoring (W=1), but the benefit saturates and eventually reverses as W grows. The underlying trade-off is that a wider window reduces the number of top-ranked slots needed to cover all dirty segments, yet simultaneously expands each slot's halo neighborhood by a factor of $2W-1$; a net benefit requires the ranking compression to outpace this geometric penalty. In our ablation, W=2 matches or improves upon W=1 in all tested configurations, with larger gains under higher collision pressure—up to ∼3.8× candidate cost reduction. Beyond W=2, the compression headroom is exhausted and halo expansion dominates when L/B is tight (full ablation in Supplement S2.4).

Contiguous vs. random grouping To isolate the contribution of contiguity, we compare two mapping topologies at W=4: *contiguous windows*, where each segment maps to W adjacent sections starting from a single hashed position, and *random grouping*, where the W sections are chosen independently and need not be adjacent. The two share identical physical budget and coverage; they differ only in spatial arrangement. Complementary evidence from the energy-coverage view is reported in Supplement Figure S4. The diff-capture curves in Figure 3 show that contiguous windows yield a markedly steeper curve: contiguous mapping concentrates the energy of dirty segments into localized peaks that are amplified by sliding-window aggregation, whereas random grouping disperses that energy and reduces separability under collision noise. This aligns with the matched-filter intuition from §4.2: random grouping breaks the coherent alignment of the contiguous support, eliminating the superposition gain of window aggregation.

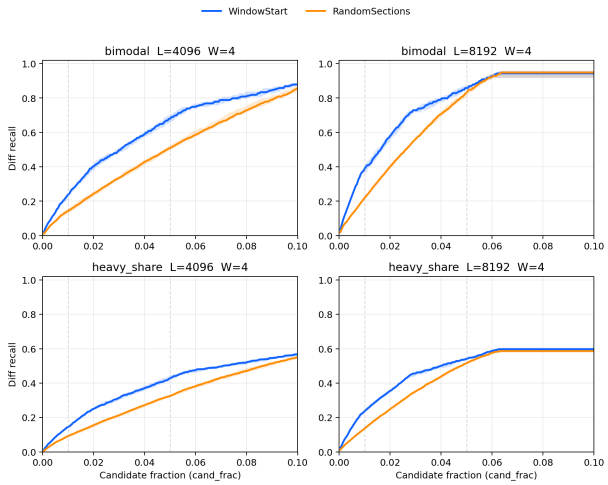


Figure 3: Diff capture curve: contiguous windows (`WindowStart`) vs. random grouping (`RandomSections`), W=4. Vertical axis: cumulative captured diff fraction. `bimodal` denotes concentrated pattern; `heavy_share` denotes mixed pattern.

The trends from both experiments are consistent with the scaling analysis in §4.2, Equations (1)–(2), although that analysis assumes sufficient signal separability. When mixed or heavy-tail noise compresses the ranking margin, the truncation quota needed to preserve recall increases, and candidate cost rises accordingly; we characterize this interaction further in §5.4. Overall, these results show that effective candidate compression depends on both multi-section support ($W > 1$) and contiguity, with the main gain coming from coherent aggregation rather than simply from increasing the number of sections touched by each segment.

5.3 De-drift Stabilizes Sizing and Gating

We first verify drift-invariance of the de-drifted statistic, then evaluate the scaling of sizing accuracy and the reliability region of three-state gating.

Stability under drift We sweep q over $0.2, 0.5, 0.9$ and compare the de-drifted energy against two controls:

`raw_sq_sum` ($\sum_j \Delta_j^2$, raw second moment, no mean removal) and `abs_net_sum` ($|\sum_j \Delta_j|$, first-order absolute moment). Table 3 reports all three statistics without bias correction.

For sizing, `raw_sq_sum` produces overestimation above 10^4% under strong drift (Table 3), whereas de-drifted energy keeps the error within about 10%; with Bessel correction this narrows further to about 3% (Table 4; Supplement Table S10).

For gating, K_{eff}/L computed from de-drifted energy remains nearly invariant across drift levels (∼0.015–0.017). In contrast, `abs_net_sum` gives similarly low values under asymmetric drift ($q \neq 0.5$) but jumps to 0.425 at q=0.5 (Table 3). At that point the concentrated fault is misread as diffuse (cross-configuration robustness in Supplement S3).

Table 3: Sizing and gating stability under drift. Top: sizing `rel_err`; bottom: gating K_{eff}/L (`heavy_share`).

Sizing (`rel_err`)	q	workload	median	p90
`raw_sq_sum`	0.2	bimodal	**+16 659%**	**17 418%**
`raw_sq_sum`	0.9	bimodal	**+29 576%**	**31 180%**
`dedrift_countscore`	0.2	bimodal	−9.1%	-4.1%
`dedrift_countscore`	0.9	bimodal	−8.9%	-3.7%
Gating (K_{eff}/L)	q		K_{eff}/L	
`dedrift_countscore`	0.2		0.0153	
`dedrift_countscore`	0.5		0.0166	
`dedrift_countscore`	0.9		0.0162	
`abs_net_sum`	0.5		**0.425**	

Predictable scaling of sizing accuracy Multi-view fusion reduces Std(rel_err) approximately as $V^{-0.5}$, consistent with the expected variance reduction from averaging near-independent views. A log-log regression across configurations yields a mean slope of −0.498 (std 0.050), confirming this $V^{-0.5}$ scaling. Increasing m_0 also reduces variance, as shown in Table 4.

Moreover, Std(rel_err) is weakly dependent on the discrepancy size D: as it spans from 10^2 to 10^7, the standard deviation remains in the approximately 2%–3% range, indicating that accuracy is governed primarily by probe capacity parameters rather than discrepancy magnitude (theoretical analysis, full curves, and stability verification in Supplement S4.3).

Table 4: Representative configurations for sizing accuracy (L=2048). Top: fixing m_0=32 and sweeping V; bottom: fixing V=4 and sweeping m_0.

Setting	Knob	Values	`Std(rel_err)` (%)
m_0=32	V	1 / 4 / 16	3.53 / 2.25 / 1.01
V=4	m_0	8 / 32 / 128	3.81 / 2.25 / 1.04

Three-state gating classification The scale separation between concentrated and diffuse regimes predicted in §4.4 requires sufficient occupancy; we now evaluate the practical reliability region of this separation using the FAR–FRR trade-off over all samples (including `unknown`).

Figure 4 shows the unconditional FAR–FRR Pareto front: each curve corresponds to a fixed τ_{occ}, and moving along a curve sweeps τ_{gating}. Higher τ_{occ} generally yields lower FAR but higher FRR. Raising the occupancy threshold excludes ambiguous samples, improving discrimination among the remainder and thus lowering FAR. However, because excluded samples are counted as `unknown` rejections in the unconditional metric, FRR rises accordingly.

Beyond $\tau_{\text{occ}} \approx 0.10$, the FRR floor settles near 13%. Two factors contribute: some genuinely diffuse samples have occupancy below the threshold and are therefore excluded as `unknown`; meanwhile, at large B (e.g., B=200, L=2048), the concentrated and diffuse regimes converge in K_{eff}/L, making them difficult to distinguish regardless of the threshold choice.

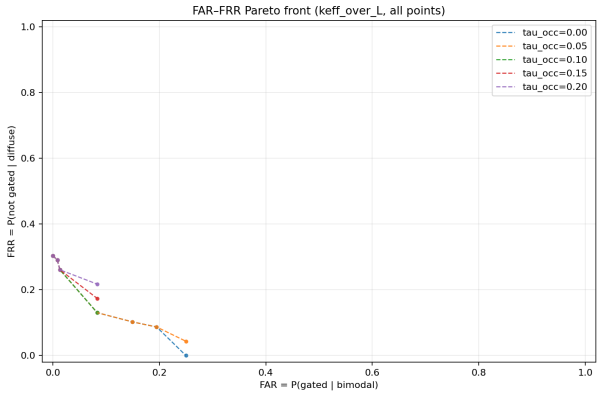


Figure 4: Unconditional FAR–FRR trade-off. Each curve corresponds to a fixed τ_{occ}; only non-dominated points are retained. Higher τ_{occ} generally yields lower FAR but higher FRR.

The preceding analysis counts all samples—including those labeled `unknown`—toward the error rates. An alternative view considers only samples that receive a decisive verdict (`notgate` or `gate`) and computes *conditional* error rates FAR_{cond} and FRR_{cond} on this subset. Under this metric, both rates can approach 0 simultaneously (Supplement Fig. S15). At τ_{occ}=0.10, approximately 87% of diffuse samples still receive a decisive classification, while most bimodal samples with small B are labeled `unknown` due to low occupancy. A two-state classifier would be forced to emit a verdict on these low-support samples despite collapsed scale separation, whereas the three-state design abstains and keeps FAR_{cond} and FRR_{cond} near zero among decisive verdicts.

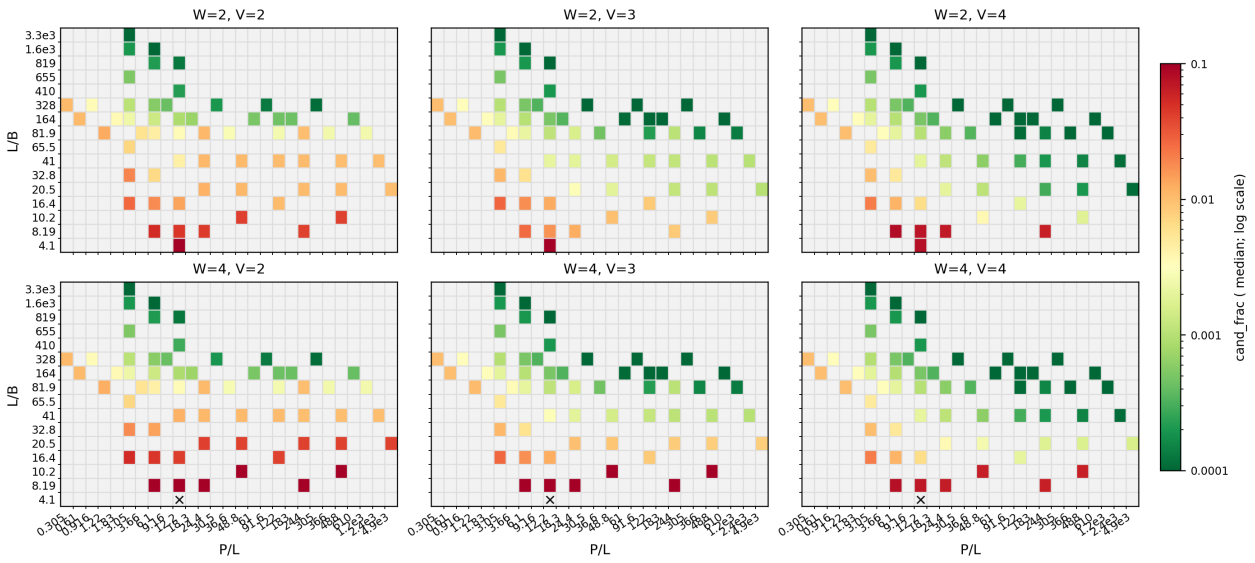


Figure 5: Operating region heatmap over P/L and L/B. Color encodes the median candidate fraction `cand_frac` on a log scale; $\times$ marks infeasible points, and gray denotes configurations with no samples.

5.4 Operating Limits and Runtime Stopping Rule

Figure 5 visualizes the system's transition from effective operation to degradation in the $(P/L,\ L/B)$ plane. The two axes govern candidate cost through distinct mechanisms.

Along decreasing L/B, the B dirty segments become denser on the section axis and their high-energy windows begin to overlap, so triage must retain more top-ranked windows to maintain recall—raising candidate cost. Equivalently, `cand_frac` decreases monotonically with increasing L/B across all configurations. This trend is visible in the per-slice curves of Supplement Figure S16, which fix P/L and sweep L/B.

As P/L increases, the larger clean-segment pool amplifies false inclusion through the $(P-B)\,p_{\text{halo}}^{V}$ term. This effect is visible even in a representative boundary slice: as P/L grows from 1.2 to 1221, candidate counts rise from the hundreds to the tens of thousands. Raising V from 2 to 4 reduces the count back to the hundreds, confirming that the P/L-driven amplification is suppressible by multi-view intersection (numerical details in Supplement Table S16).

Under a fixed memory budget $M_{\text{bytes}} = 4VLm_0$, adding views forces L to shrink proportionally ($L \propto 1/V$). More views strengthen cross-view intersection suppression exponentially, but the smaller L raises the per-view halo probability $p_{\text{halo}} \approx t(2W-1)/L$, where $t \approx \alpha BW$ and $\alpha = O(1)$ is a truncation factor. This creates a fundamental trade-off: once L/B drops below a threshold, p_{halo} approaches 1, cross-view intersection loses its filtering power, and candidate cost rises sharply. A closed-form analysis (Supplement S5.2) yields a sufficient condition for the intersection mechanism to remain effective: $L/B \gtrsim e\alpha W(2W-1)$, which evaluates to $L/B \approx 16$ for W=2, α=1—the order of magnitude for the transition band visible in Figure 5. Above this region, the P/L-driven amplification can be absorbed exponentially by multi-view intersection, and degradation along that axis is considerably milder.

Runtime stopping rule In §4.2, triage ranks windows by energy and selects the top $t = \lceil \mathrm{r}_{\text{top}} \cdot (L-W+1) \rceil$; larger r_{top} yields a larger candidate set. Since B is unknown at runtime and may exceed the planning prior B^*, SCOUT applies a runtime truncation rule. A purely observable signal provides this adaptation. As r_{top} is swept upward during local decoding (a negligible in-memory computation once the probe is built), the system tracks how much energy the selected sections capture and how many candidate segments they produce.

Figure 6 plots both quantities against cumulative energy fraction for a representative configuration. The first few top-ranked sections capture the bulk of the energy while candidate cost remains low; beyond a knee, additional sections contribute diminishing energy but candidate count accelerates sharply. This synchronized knee provides a natural stopping signal that requires no knowledge of the true B: expansion halts when marginal energy gain per unit increase in candidate cost drops (implementation details in Supplement S5.3). At the knee (r_{top}=0.25), the cutoff captures approximately 85% of total energy at a candidate fraction under 2%. Doubling r_{top} beyond this point yields less than 5 percentage points of additional energy but increases candidate count by more than an order of magnitude. Using ground-truth recall for offline verification, we find that this knee marks the point where additional candidate cost yields only modest recall

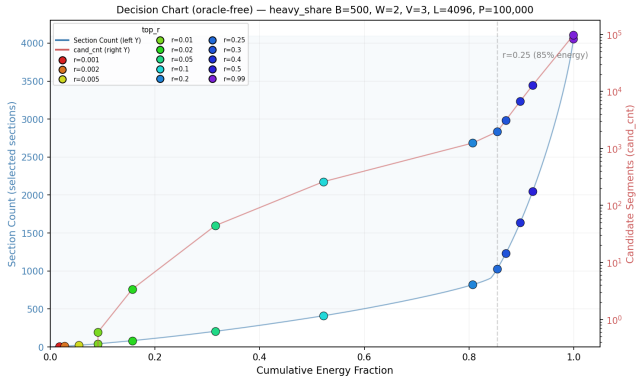


Figure 6: Selected sections (left axis) and candidate cost (right axis, log scale) plotted against cumulative energy fraction along the `top_r` schedule. The two curves share a synchronized knee.

gains (Supplement Fig. S19).

5.5 Same-Budget Comparison and Failure Regions

Under a common sketch budget of approximately 512 KiB, we compare SCOUT, B2, and B3, focusing on the regimes in which triage and gating begin to fail.

Triage degrades under heavy-tail noise We construct a family of mixed workloads with P=100,000 and `total_diff`=200,000, comprising B heavily dirty segments (each with d_{big} diff keys) and S noise segments (each with d_{small} diff keys). The difficulty of separating heavy hitters from noise is parameterized along two axes: `S/B` (noise-to-signal multiplier) and `ratio`=$d_{\text{big}}/d_{\text{small}}$ (separability). Figure 7 shows each method's feasible region on this two-dimensional parameter grid under a 2% candidate budget cap. Greener cells indicate a lower required candidate fraction, whereas red cells marked as N/A denote configurations that cannot meet the recall constraint.

B2 degrades when heavy-tail noise collapses the per-segment ranking margin. With many noise segments (high `S/B`) and a small intensity gap (low `ratio`), mildly dirty segments flood the top-K ranking and displace the genuinely heavy ones. At high separability (`ratio`=200), B2 achieves a slightly lower candidate fraction than SCOUT ($\sim 10^{-4}$ vs. $\sim 2\times 10^{-4}$), because direct ranking is highly effective when per-segment signals are already well separated; SCOUT pays a small halo cost from window aggregation. Once `ratio` drops to single digits, however, B2 rapidly enters N/A. Doubling B2's memory to $\sim$1 MiB does not change this picture: the low-separability region at `ratio` ≤ 5 remains unreachable (Supplement S6.2).

B3 is limited by the discrete granularity of its hierarchy. Once a leaf bucket is selected, the entire bucket must be expanded, so the candidate fraction increases in bucket-sized jumps rather than smoothly. Under heavy-tail noise, many buckets accumulate moderate discrepancy mass at the same time, which pushes the candidate set quickly toward the 2% cap. An ablation over B (10→100) shows that SCOUT's feasible region shrinks by 25 pp, compared with 43 pp for B2 and 39 pp for B3. Additional checks, including doubling B2's memory, relaxing the candidate cap from 2% to 5% and 10%, and sweeping B, do not change the qualitative comparison (Supplement S6.2).

Gating reliability under the same budget Figure 8 compares the conditional FAR–FRR trade-off of the three methods at $\sim$512 KiB, treating `gate` as the positive class and conditioning on decisive (that is, non-`unknown`) samples. Full definitions, threshold sweeps, and supplementary analyses are provided in Supplement S6.3. SCOUT uses K_{eff}/L as the gating statistic. B2 uses the fraction of total estimated mass not captured by its top-ranked segments, which stays near 0 when discrepancy mass is concentrated in a few segments and increases as the mass becomes more diffuse. B3 uses the maximum per-level fan-out ratio, using the most expanded level as a proxy for how diffusely the discrepancy spreads through the hierarchy.

Only SCOUT attains operating points with both low FAR and low FRR. Under the constraint `FRR_cond` ≤ 0.10, τ_{gating} remains feasible across the interval 0.15–0.40; at the representative point τ_{gating}=0.40, `FAR_cond` ≈ 0.038 and `FRR_cond`=0.10. The residual $\sim$4% conditional false-alarm rate is concentrated in borderline cases with high noise coverage. In contrast, B2's curve plateaus at `FRR_cond`=0.50: regardless of threshold tuning, half of the diffuse samples are never intercepted, because heavy-tail noise dilutes concentrated faults to resemble diffuse ones, and B2's statistic cannot distinguish the two. B3 remains confined to the high-FAR, high-FRR region. Nearly all concentrated faults are misclassified as diffuse. Within shared buckets, collisions between dirty and clean segments inflate upper-level fan-out even for concentrated faults, leaving B3 with no usable trade-off interval (Supplement S6.3).

5.6 Validation under YCSB-Driven Workloads

This section introduces segment-size skew as a robustness dimension absent from the synthetic experiments above. We use a YCSB Zipfian distribution (α=0.99) to generate the base keyspace and evaluate the following fault families: *Bimodal* (bounded-blast-radius concentrated faults), *Drift-shaped* (lag concentrated on hot segments), and *Uniform Diffuse* (broad low-rate anomalies). Each family is instantiated at three data-scale tiers—Light (20M records, P=100,000), Medium (200M), and Heavy (2B)—which induce progressively denser segment fill states under the same Zipfian workload. A fourth

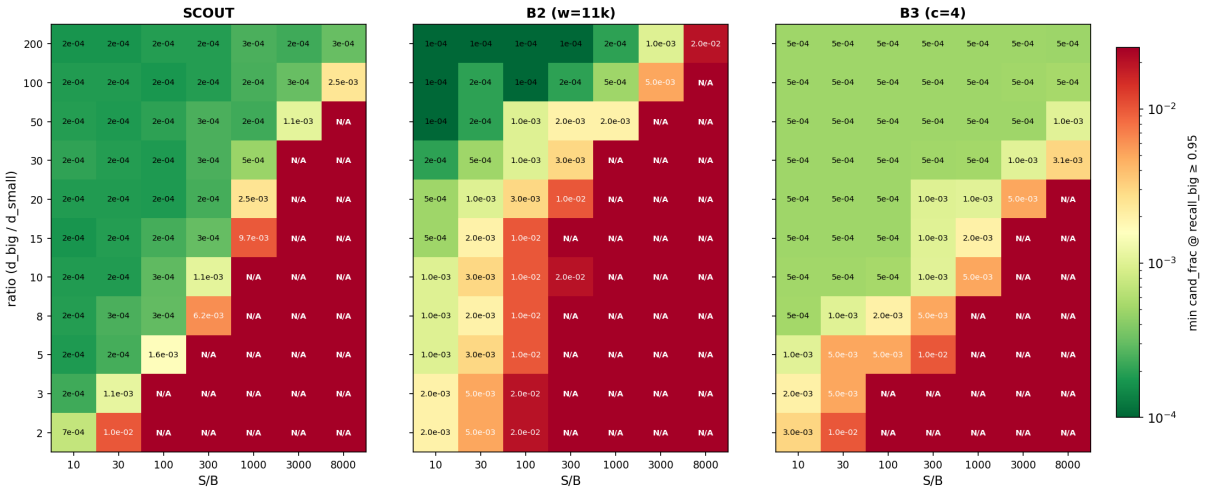


Figure 7: Triage operating-point heatmap (same budget, single-round probe). Color encodes the minimum `cand_frac` achieving `median(recall_big)` ≥ 0.95 (log scale); N/A indicates the recall constraint cannot be met within the 2% cap.

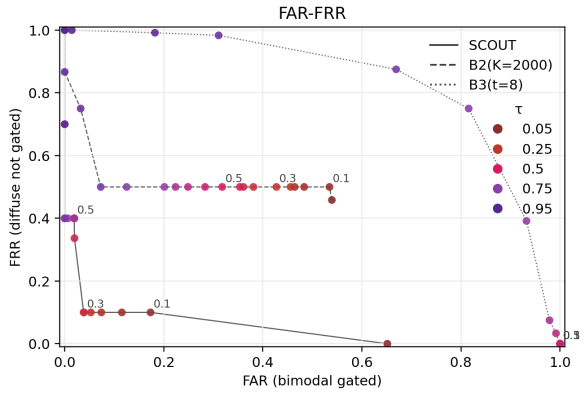


Figure 8: Conditional FAR–FRR trade-off (∼512 KiB). Each point on a curve corresponds to a τ_{gating} threshold; the lower-left corner is the ideal region.

family, *Weighted Diffuse* (per-record loss under segment-size skew), is further detailed in Supplement S7 along with parameterization details for all families (Supplement Tables S26 and S25).

Gating The upper panel of Table 5 reports SCOUT's K_{eff}/L, as §5.5 shows that neither baseline provides a reliable gating signal. All four fault families form a clear gradient: Bimodal at approximately 0.003 (highly concentrated), Drift-shaped at approximately 0.08, Weighted Diffuse at approximately 0.12, and Uniform Diffuse at approximately 0.93 (highly diffuse). The wide separation between the first three families ($K_{\text{eff}}/L \leq 0.12$) and Uniform Diffuse ($\geq$0.93) leaves a broad margin for placing τ_{gating}: the first three families are admitted to the triage path while Uniform Diffuse is diverted to fallback. K_{eff}/L is nearly invariant across the three data-scale tiers, indicating that the gating metric is governed by segment-level fault shape rather than data density. All families yield sufficient occupancy for decisive gating across all three tiers, with no `unknown` outcomes; full distributions are shown in Supplement Figure S30.

Triage The lower panel of Table 5 reports the minimum candidate fraction achieving `recall_big` ≥ 0.95 under a 2% cap. Bimodal faults yield a stable candidate fraction of 3×10^{-5}; Drift-shaped faults yield approximately 7.3×10^{-4}. Across the three tiers, candidate fractions change little: with P and the segment-level fault pattern held fixed, increasing the record count mainly densifies segment fill rather than changing the sketch-level concentration pattern. Uniform Diffuse does not appear in the triage panel because gating diverts it to fallback.

Sizing Drift-shaped faults are the hardest sizing case because they combine segment-size skew with concentration on hot segments. With the Bessel correction applied, the de-drifted estimator achieves a median relative error of 0.8%–1.4% across the three tiers (p90 $\leq$ 2.8%), matching the accuracy observed under the uniform settings of §5.3; raw-control results are reported in Supplement Table S27.

Sections 5.2–5.6 evaluate SCOUT's triage, sizing, and gating behavior across controlled synthetic workloads and YCSB-driven skewed workloads. Because these results assume uniform hashing at the probe interface, we additionally verify in Supplement S8 that the triage operating points remain stable across three hash implementations, two key structures, and $k \in \{1,2,3\}$.

Table 5: YCSB-driven validation summary across data-scale tiers. Upper: gating statistic; lower: triage candidate fraction.

Family	Light	Medium	Heavy
SCOUT Gating: median K_{eff}/L			
Bimodal	0.003	0.003	0.003
Drift-shaped	0.082	0.082	0.081
Weighted Diffuse	0.122	0.118	0.120
Uniform Diffuse	0.931	0.938	0.939
SCOUT Triage: min `cand_frac` at `recall_big`$\geq$0.95			
Bimodal	3×10^{-5}	3×10^{-5}	3×10^{-5}
Drift-shaped	7.3×10^{-4}	7.4×10^{-4}	7.3×10^{-4}

6 Discussion

SCOUT is designed for deployments where reconciliation runs as a continuous background function, most rounds are expected to be clean, and exact verification is expensive relative to a fixed-size probe exchange. In this setting, the three probe outputs serve distinct operational roles: sizing always provides a severity signal; gating decides whether the current discrepancy shape supports economical candidate compression; and triage narrows the verification scope when the discrepancy is sufficiently concentrated. Table 6 summarizes the round-level interpretation. In the common clean-round case, reconciliation terminates after a single fixed-cost probe exchange, without invoking downstream verification. Precise localization and repair within the candidate scope remain delegated to downstream mechanisms such as Merkle trees or IBLTs; SCOUT's role ends at diagnosis.

Table 6: Representative operational scenarios and typical probe outcomes.

Scenario	$\hat{D}$	g	Action
Normal round	≈ 0	–	skip downstream verification
Shard failure	varies	`notgate`	triage, then verify on $\mathcal{C}$
One-sided backlog	large	varies	severity alert; triage if shape admits, else fallback
Diffuse discrepancy	moderate or large	`gate`	bypass triage; use fallback
Low observability	varies	`unknown`	defer, capped triage, or fallback
Segmenter misconfiguration	$\approx$ total	–	inspect config

Among the probe outcomes in Table 6, `unknown` merits separate discussion. It is an explicit abstention: the probe lacks sufficient statistical support for a reliable shape judgment. The most common trigger is a small number of concurrent faults, which leaves too few sections energized for K_{eff}/L to reliably separate concentrated from diffuse patterns. Paradoxically, this is also the regime where triage is most effective, since a small B yields high signal separability and strong candidate compression. In deployment, `unknown` rounds can still be acted upon by combining $\hat{D}$ with other observables such as occupancy: deferring when $\hat{D}$ is small, or proceeding with a capped triage pass when $\hat{D}$ is large. What SCOUT withholds is a shape-based recommendation, not the scale signal itself.

A natural question is whether the discrepancy axes used in our evaluation—concentration, diffuseness, and side imbalance—provide a useful abstraction for fault scenarios encountered in practice. Our results suggest that they capture several common manifestations relevant to continuous reconciliation: concentrated discrepancies tend to benefit from triage, diffuse ones are more likely to be diverted by gating, and asymmetric ones still admit a stable sizing signal through de-drifted energy. This abstraction is not intended as an exhaustive fault taxonomy. One scenario that may appear to fall outside it is spatially correlated dirty segments in the underlying key space; however, after hash-based segment mapping, such correlation often appears as a larger effective B, which is already reflected in the capacity-degradation dimension characterized in §5.4.

Two limitations remain. First, the synthetic and YCSB-driven workloads do not establish how frequently each discrepancy class arises in production, nor whether additional classes emerge under real operational correlations or temporal patterns. Second, under extreme multi-fault overlap ($B \cdot W$ approaching L), the sparse-overlap approximation used in the candidate-size analysis begins to break down, and this regime would benefit from a tighter characterization.

7 Conclusion

SCOUT is a single-round, fixed-budget diagnostic probe that serves as a front-end to precise downstream verification in heterogeneous replication. From one sketch exchange, it jointly provides discrepancy sizing, shape-based gating, and candidate-scope triage, all derived from a shared de-drifted statistical substrate. Across synthetic stress tests, same-budget baseline comparisons, and YCSB-driven workloads, the results show that these three outputs can be combined within a single fixed-budget probe while remaining useful across concentrated, diffuse, and asymmetric discrepancy regimes.

More broadly, SCOUT shows that continuous reconciliation need not pay full validation cost on every round. By separating diagnosis from exact repair, clean rounds terminate after a fixed probe exchange, while anomalous rounds invoke more expensive downstream verification only when the observed discrepancy warrants it. An important next step is to validate how the discrepancy classes studied here are distributed in production traces and deployments.

References

[1] Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO '87*, volume 293 of *Lecture Notes in Computer Science*, pages 369–378, Berlin, Heidelberg, 1988. Springer.

[2] Alan Demers, Dan Greene, Carl Hauser, Wes Irish, John Larson, Scott Shenker, Howard Sturgis, Dan Swinehart, and Doug Terry. Epidemic algorithms for replicated database maintenance. In *Proceedings of the 6th Annual ACM Symposium on Principles of Distributed Computing (PODC)*, pages 1–12, Vancouver, British Columbia, Canada, 1987. ACM.

[3] Robbert van Renesse, Yaron Minsky, and Mark Hayden. A gossip-style failure detection service. In *Proceedings of the IFIP International Conference on Distributed Systems Platforms (Middleware)*, pages 55–70. Springer, 1998.

[4] Robbert van Renesse, Dan Mihai Dumitriu, Valient Gough, and Chris Thomas. Efficient reconciliation and flow control for anti-entropy protocols. In *Proceedings of the 2nd Workshop on Large-Scale Distributed Systems and Middleware (LADIS)*. ACM, 2008.

[5] Werner Vogels. Eventually consistent. *Communications of the ACM*, 52(1):40–44, 2009.

[6] Douglas B Terry, Marvin M Theimer, Karin Petersen, Alan J Demers, Mike J Spreitzer, and Carl H Hauser. Managing update conflicts in a weakly connected replicated storage system. In *Proceedings of the fifteenth ACM symposium on Operating systems principles (SOSP)*, pages 172–182, 1995.

[7] Yaron Minsky, Ari Trachtenberg, and Richard Zippel. Set reconciliation with nearly optimal communication complexity. *IEEE Transactions on Information Theory*, 49(9):2213–2218, 2003.

[8] David Eppstein, Michael T. Goodrich, Frank Uyeda, and George Varghese. What's the difference? Efficient set reconciliation without prior context. In *Proceedings of the ACM SIGCOMM 2011 Conference*, pages 218–229, Toronto, Ontario, Canada, 2011. ACM.

[9] Michael T. Goodrich and Michael Mitzenmacher. Invertible Bloom lookup tables. In *Proceedings of the 49th Annual Allerton Conference on Communication, Control, and Computing*, pages 792–799. IEEE, 2011.

[10] Matthew Aldridge, Oliver Johnson, and Jonathan Scarlett. Group testing: An information theory perspective. *Foundations and Trends in Communications and Information Theory*, 15(3-4):196–392, 2019.

[11] Long Gong, Sourav Sahoo, Leron Drain, and Jun Xu. Space- and computationally-efficient set reconciliation via parity bitmap sketch (PBS). *Proceedings of the VLDB Endowment*, 14(4):458–470, 2021.

[12] Lei Yang, Yossi Gilad, and Mohammad Alizadeh. Practical rateless set reconciliation. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 483–499, Sydney, NSW, Australia, 2024. ACM.

[13] Nils Fleischhacker, Kasper Green Larsen, Maciej Obremski, and Mark Simkin. Invertible Bloom lookup tables with less memory and randomness. In *Proc. 32nd Annual European Symposium on Algorithms (ESA)*, volume 308 of *LIPIcs*, pages 54:1–54:17, 2024.

[14] Djamal Belazzougui, Gregory Kucherov, and Stefan Walzer. Better space-time-robustness trade-offs for set reconciliation. In *Proc. 51st International Colloquium on Automata, Languages, and Programming (ICALP)*, volume 297 of *LIPIcs*, pages 20:1–20:17, 2024.

[15] Tomer Keniagin, Eitan Yaakobi, and Ori Rottenstreich. Certainsync: Rateless set reconciliation with certainty. *Proc. ACM Meas. Anal. Comput. Syst.*, 9(2), June 2025.

[16] Burton H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.

[17] Bin Fan, Dave G. Andersen, Michael Kaminsky, and Michael D. Mitzenmacher. Cuckoo filter: Practically better than Bloom. In *Proceedings of the 10th ACM International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, pages 75–88. ACM, 2014.

[18] Peter C. Dillinger and Stefan Walzer. Ribbon filter: practically smaller than bloom and xor. arXiv preprint, 2021.

[19] Andrei Broder and Michael Mitzenmacher. Network applications of Bloom filters: A survey. *Internet Mathematics*, 1(4):485–509, 2004.

[20] Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. HyperLogLog: The analysis of a near-optimal cardinality estimation algorithm. In *Proceedings of the 2007 Conference on Analysis of Algorithms (AofA)*, volume AH of *Discrete Mathematics and Theoretical Computer Science Proceedings*, pages 127–146, 2007.

[21] Stefan Heule, Marc Nunkesser, and Alexander Hall. HyperLogLog in practice: Algorithmic engineering of a state of the art cardinality estimation algorithm. In *Proceedings of the 16th International Conference on Extending Database Technology (EDBT/ICDT)*, pages 683–692, 2013.

[22] Salvatore Sanfilippo. Redis new data structure: the HyperLogLog. http://antirez.com/news/75, 2014. Accessed: 2026-02.

[23] Apache Software Foundation. Apache DataSketches: Frequent items sketches. https://datasketches.apache.org/docs/Frequency/FrequencySketches.html, 2024. Accessed: 2026-02.

[24] Otmar Ertl. SetSketch: Filling the gap between MinHash and HyperLogLog. *Proceedings of the VLDB Endowment*, 14(11):2244–2257, 2021.

[25] Michael Mitzenmacher, Rasmus Pagh, and Ninh Pham. Efficient estimation for high similarities using odd sketches. In *Proceedings of the 23rd International Conference on World Wide Web (WWW)*, pages 109–118, 2014.

[26] Moses Charikar, Kevin Chen, and Martin Farach-Colton. Finding frequent items in data streams. In *Proceedings of the 29th International Colloquium on Automata, Languages and Programming (ICALP)*, volume 2380 of *Lecture Notes in Computer Science*, pages 693–703, Berlin, Heidelberg, 2002. Springer.

[27] Graham Cormode and S. Muthukrishnan. An improved data stream summary: The count-min sketch and its applications. *Journal of Algorithms*, 55(1):58–75, 2005.

[28] Graham Cormode and Marios Hadjieleftheriou. Finding frequent items in data streams. *Proceedings of the VLDB Endowment*, 1(2):1530–1541, 2008.

[29] Cristian Estan and George Varghese. New directions in traffic measurement and accounting. In *Proceedings of the ACM SIGCOMM Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications*, pages 323–336. ACM, 2002.

[30] Minlan Yu, Lavanya Jose, and Rui Miao. Software defined traffic measurement with OpenSketch. In *Proceedings of the 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 29–42. USENIX Association, 2013.

[31] Úlfar Erlingsson, Vasyl Pihur, and Aleksandra Korolova. Rappor: Randomized aggregatable privacy-preserving ordinal response. In *Proceedings of the 2014 ACM SIGSAC conference on computer and communications security*, pages 1054–1067, 2014.

[32] Graham Cormode, Somesh Jha, Tejas Kulkarni, Ninghui Li, Divesh Srivastava, and Tianhao Wang. Privacy at scale: Local differential privacy in practice. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pages 1655–1658. ACM, 2018.

[33] Hui Han, Zheng Yan, Xuyang Jing, and Witold Pedrycz. Applications of sketches in network traffic measurement: A survey. *Information Fusion*, 82:58–85, 2022.

[34] Chen-Yu Hsu, Piotr Indyk, Dina Katabi, and Ali Vakilian. Learning-based frequency estimation algorithms. In *International Conference on Learning Representations (ICLR)*, 2019.

[35] Shrinivas Kudekar, Thomas J. Richardson, and Rüdiger L. Urbanke. Threshold saturation via spatial coupling: Why convolutional LDPC ensembles perform so well over the BEC. *IEEE Transactions on Information Theory*, 57(2):803–834, 2011.

[36] Amin Coja-Oghlan, Oliver Gebhard, Max Hahn-Klimroth, and Philipp Loick. Optimal group testing. In Jacob Abernethy and Shivani Agarwal, editors, *Proceedings of Thirty Third Conference on Learning Theory*, volume 125 of *Proceedings of Machine Learning Research*, pages 1374–1388. PMLR, 09–12 Jul 2020.

[37] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. Dynamo: Amazon's highly available key-value store. In *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP)*, pages 205–220, Stevenson, Washington, USA, 2007. ACM.

[38] Avinash Lakshman and Prashant Malik. Cassandra: A decentralized structured storage system. *ACM SIGOPS Operating Systems Review*, 44(2):35–40, 2010.

[39] Ricardo Jorge Tomé Gonçalves, Paulo Sérgio Almeida, Carlos Baquero, and Vitor Fonte. Dotteddb: Anti-entropy without merkle trees, deletes without tombstones. In *2017 IEEE 36th Symposium on Reliable Distributed Systems (SRDS)*, pages 194–203. IEEE, 2017.

[40] Rebecca Taft, Irfan Sharif, Andrei Matei, Nathan VanBenschoten, Jordan Lewis, Tobias Grieger, Kai Niemi, Andy Woods, Anne Birzin, Raphael Poss, Paul Bardea, Amruta Ranade, Ben Darnell, Bram Gruneir, Justin Jaffray, Lucy Zhang, and Peter Mattis. CockroachDB: The resilient geo-distributed SQL database. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 1493–1509. ACM, 2020.

[41] Wyatt Lloyd, Michael J Freedman, Michael Kaminsky, and David G Andersen. Don't settle for eventual: scalable causal consistency for wide-area storage with COPS. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles (SOSP)*, pages 401–416, 2011.

[42] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Proceedings of the 13th International Conference on Stabilization, Safety, and Security of Distributed Systems*, SSS'11, pages 386–400, Berlin, Heidelberg, 2011. Springer-Verlag.

[43] Percona LLC. pt-table-checksum — Percona toolkit documentation. https://docs.percona.com/percona-toolkit/pt-table-checksum.html, 2024. Accessed 2026-02.

[44] Amazon Web Services. Aws dms data validation. https://docs.aws.amazon.com/dms/latest/userguide/CHAP_Validating.html, 2024. Accessed 2026-02.

[45] Google Cloud. Verify a migration. https://docs.cloud.google.com/database-migration/docs/sqlserver-to-csql-pgsql/verify-migration, 2024. Accessed 2026-02.

[46] Cockroach Labs. Molt verify. https://www.cockroachlabs.com/docs/molt/molt-verify, 2026. Accessed 2026-02.